

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO ĐỒ ÁN MÔN HỌC
Toán ứng dụng và thống kê

Giáo viên hướng dẫn: ThS. Lê Nhựt Nam
ThS. Võ Nam Thực Đoan

Nhóm thực hiện: Nhóm 3 - Lớp 24CTT2
Thành viên: 24120018 – Đào Thị Quỳnh Anh
24120347 - Phan Lê Đăng Khoa
24120348 – Hà Đăng Khôi
24120352 – Lương Trung Kiên
24120418 – Nguyễn Minh Quân

Hồ Chí Minh, 2026

Mục lục

Bảng phân công công việc	5
1 Phần 1: Phép khử Gauss và Các Ứng Dụng	7
1.1 Cơ sở lý thuyết	7
1.2 Cài đặt thuật toán	7
1.3 Kiểm thử và đối chiếu kết quả	7
1.3.1 Phương pháp đo lường sai số	7
1.3.2 Kịch bản kiểm thử và kết quả thực nghiệm	7
1.3.3 Phân tích các vấn đề và hiện tượng sai số	8
2 Phần 2: Phân Rã Ma Trận và Trục Quan Hóa với Manim	10
2.1 Cơ sở lý thuyết phương pháp phân rã SVD	10
2.1.1 Định nghĩa và Định lý tồn tại	10
2.1.2 Mối liên hệ với Trị riêng và Vector riêng	10
2.1.3 Ý nghĩa hình học của SVD	10
2.1.4 Xấp xỉ hạng thấp và Định lý Eckart-Young	11
2.2 Cài đặt thuật toán SVD trong Python	11
2.2.1 Tổng quan luồng thuật toán	11
2.2.2 Module cấu hình toàn cục: config.py	11
2.2.3 Các hàm tiện ích nội bộ (private)	12
2.2.4 Thuật toán Jacobi tìm trị riêng: _jacobi_eigen_symmetric	14
2.2.5 Hàm chính: decompose_svd(A)	15
2.2.6 Kiểm thử hàm decompose_svd	17
2.3 Chéo hóa ma trận	17
2.3.1 Tổng quan	17
2.3.2 Hàm phân rã QR: _qr_decomposition	17
2.3.3 Thuật toán lặp QR tìm trị riêng: _qr_eigen_diagonal	18
2.3.4 Nhóm trị riêng: _group_eigenvalues	19
2.3.5 Không gian null và vector riêng: _rref và _null_space_basis	19
2.3.6 Hàm chính: diagonalize(A)	19
2.3.7 Kiểm thử hàm diagonalize	21
2.4 Kịch bản trục quan hóa Manim	21
2.4.1 Tổng quan video	21
2.4.2 Cảnh 1: Giới thiệu bài toán	22
2.4.3 Cảnh 2: Biến đổi V^T (Quay lần 1)	22

2.4.4	Cảnh 3: Biến đổi Σ (Co giãn)	22
2.4.5	Cảnh 4: Biến đổi U (Quay lần 2)	22
2.4.6	Cảnh 5: Tổng kết và kết luận	22
3	Phần 3: Giải Hệ Phương Trình và Phân Tích Hiệu Năng	23
3.1	Thiết lập thực nghiệm	23
3.1.1	Cấu hình phần cứng và phần mềm	23
3.1.2	Môi trường kiểm thử và phương pháp đo lường	23
3.1.3	Chiến lược cài đặt Solver và cơ chế Fallback	23
3.2	Phân tích hiệu năng	24
3.2.1	Cơ sở lý thuyết về độ phức tạp thuật toán	24
3.2.2	Kết quả đo lường thời gian thực thi	24
3.2.3	Nhận xét và đối chiếu lý thuyết	24
3.3	Phân tích độ ổn định số học	26
3.3.1	Cơ sở lý thuyết: Số điều kiện và định lý sai số	26
3.3.2	Thiết lập hai đối tượng khảo sát	26
3.3.3	Kết quả đo lường sai số tương đối	27
3.3.4	Nhận xét và lý giải hiện tượng	27
3.4	Tổng kết và đánh giá lựa chọn phương pháp	28
4	Kết luận	30
4.1	Tóm tắt kết quả đạt được	30
4.2	Bài học rút ra	31
4.3	Khó khăn chuyên môn và các nghịch lý Toán học tính toán	31
A	Phụ lục	35
A.1	Bảng số liệu chi tiết	35
A.2	Code bổ sung	35

Danh sách hình vẽ

- 1 Đánh giá độ ổn định số học: Custom Gaussian vs NumPy 9
- 2 Đồ thị Log-Log so sánh thời gian thực thi của ba phương pháp theo kích thước n . Đường nét đứt là các đường tham chiếu lý thuyết $y \propto n^3$ và $y \propto n^2$ 25

Danh sách bảng

1	Bảng phân công công việc và mức độ đóng góp	5
2	Thời gian trung bình thực thi (giây) theo kích thước n trên ma trận SPD	25
3	So sánh số điều kiện $\kappa_2(A)$ và sai số tương đối $\ Ax - b\ _2/\ b\ _2$	27
4	Bảng so sánh tổng hợp ba phương pháp giải hệ phương trình tuyến tính	29
5	Bảng số liệu chi tiết	35

Danh mục mã nguồn

1	code mẫu	7
2	Hàm xử lý sai số	12
3	Hàm trực giao hóa Gram-Schmidt	13
4	Hàm xây dựng ma trận vector riêng	14
5	Hàm phân rã SVD	16
6	Hàm phân rã QR	17
7	Hàm lặp QR tìm trị riêng	18
8	Hàm chéo hóa ma trận	20
9	Mã nguồn xử lý dữ liệu (Python)	35

BẢNG PHÂN CÔNG CÔNG VIỆC VÀ MỨC ĐỘ ĐÓNG GÓP

Bảng 1: Bảng phân công công việc và mức độ đóng góp

STT	Họ tên / MSSV	Nhiệm vụ đảm nhiệm chi tiết	Đóng góp
1	24120018 Đào Thị Quỳnh Anh	- Biên soạn <code>part1_demo.ipynb</code> minh họa trực quan kết quả Phần 1. - Xây dựng <code>data_gen.py</code>: Viết script sinh ma trận ngẫu nhiên SPD (well-conditioned) và ma trận Hilbert H_n (ill-conditioned). - Phân tích tính ổn định: Tính số điều kiện $\kappa_2(A)$, thu thập sai số tương đối và lý luận giải thích hiện tượng theo Định lý 3.1.	100%
2	24120347 Phan Lê Đăng Khoa	- Cài đặt thuật toán phân rã SVD (Phần 2). - Xây dựng <code>solvers.py</code>: Tích hợp phương pháp Gauss, SVD và cài đặt thuật toán lặp Gauss-Seidel (kèm hàm kiểm tra ma trận chéo trội). - Xây dựng <code>benchmark.py</code>: Lên kịch bản đo thời gian thực thi (trung bình 5 vòng lặp), tính sai số tương đối và xuất dữ liệu ra định dạng JSON chuẩn.	100%
3	24120348 Hà Đăng Khôi	- Trực quan hóa Toán học (Phần 2): Viết kịch bản và lập trình tạo Video demo hoạt ảnh quá trình phân rã ma trận bằng thư viện Manim. - Phối hợp viết báo cáo tổng kết, biên tập tài liệu và quay dựng thành phẩm nộp bài cuối cùng.	100%

STT	Họ tên / MSSV	Nhiệm vụ đảm nhiệm chi tiết	Đóng góp
4	24120352 Lương Trung Kiên	• Phối hợp cài đặt thuật toán Phép khử Gauss và ứng dụng (Phần 1). • Xây dựng <code>analysis.ipynb</code> (Phần 3): Đọc file dữ liệu JSON, vẽ đồ thị log-log so sánh thời gian thực tế với đường lý thuyết $O(n^3)$. • Trình bày Markdown lý thuyết Toán học (số điều kiện, Gauss-Seidel) và viết kết luận đánh giá tương quan giữa tính ổn định và chi phí tính toán.	100%
5	24120418 Nguyễn Minh Quân	• Phối hợp cài đặt thuật toán Phép khử Gauss và ứng dụng (Phần 1). • Biên soạn Báo cáo bằng $\text{\LaTeX}$(<code>report.tex</code>): Dựng khung tài liệu, xử lý các môi trường công thức Toán học phức tạp cho Phần 1 và Phần 2. • Trích xuất dữ liệu, chèn biểu đồ từ Notebook vào báo cáo, viết phần Mở đầu, Kết luận và rà soát toàn bộ source code trước khi nộp.	100%

1 Phần 1: Phép khử Gauss và Các Ứng Dụng

Trong phần này, nhóm tiến hành cài đặt phương pháp khử Gauss có chọn phần tử chốt (Partial Pivoting) từ đầu bằng Python...

1.1 Cơ sở lý thuyết

Nội dung cơ sở lý thuyết về phép khử Gauss, định nghĩa ma trận tăng cường, các phép biến đổi dòng sơ cấp...

1.2 Cài đặt thuật toán

Trình bày về cấu trúc hàm `gaussian_eliminate(A, b)`...

```
1 import numpy as np
2
3 def process_data(data_path):
4     print("Code minh hoa chen tai phu luc...")
5     return True
```

Đoạn mã 1: code mẫu

1.3 Kiểm thử và đối chiếu kết quả

1.3.1 Phương pháp đo lường sai số

Để đánh giá tính đúng đắn và độ ổn định của thuật toán Khử Gauss tự cài đặt (Custom Solver), nhóm không chỉ so sánh trực tiếp vector nghiệm x thu được với kết quả từ thư viện `numpy.linalg.solve`, mà còn sử dụng **Chuẩn L2 của vector phần dư (Residual Norm)** làm thước đo chính:

$$e = \|r\|_2 = \|b - Ax\|_2 = \sqrt{\sum_{i=1}^n (b_i - (Ax)_i)^2} \quad (1)$$

Việc đo lường sai số phần dư giúp phản ánh chính xác chất lượng của nghiệm đối với cấu trúc ma trận A ban đầu, đặc biệt hữu ích khi đánh giá các hệ phương trình nhạy cảm với nhiễu.

1.3.2 Kịch bản kiểm thử và kết quả thực nghiệm

Nhóm đã xây dựng một bộ Test Suite gồm 7 kịch bản, bao phủ từ các trường hợp cơ bản đến các "điểm chết" của chuẩn biểu diễn dấu phẩy động IEEE 754 (Double Precision).

STT	Kịch bản kiểm thử (Test Case)	Ý nghĩa kiểm tra	Sai số L2 (e)
1	Hệ cơ bản (Nghiệm nguyên)	Tính đúng đắn của thuật toán	$0.00e + 00$
2	Nhiều làm tròn (Round-off)	Phép chia cho số vô hạn tuần hoàn	$5.55e - 17$
3	Cần Partial Pivoting	Khử phần tử chốt (pivot) tiệm cận 0	$0.00e + 00$
4	Chênh lệch hệ số khổng lồ	Hiện tượng nuốt số (Swamping)	$0.00e + 00$
5	Hệ điều kiện kém (Near-singular)	Khuếch đại sai số do hai dòng gần song song	$2.22e - 16$
6	Ma trận Hilbert 3x3	Ma trận có số điều kiện lớn	$1.11e - 16$
7	Ma trận suy biến	Khả năng bắt lỗi hệ vô số nghiệm/vô nghiệm	Không có nghiệm

1.3.3 Phân tích các vấn đề và hiện tượng sai số

Dựa trên kết quả thực nghiệm, nhóm rút ra các phân tích sâu sắc về bản chất của tính toán số học trên máy tính:

a) Vấn đề 1: Nhiều làm tròn số

Ở Kịch bản 1, sai số trả về là 0.00. Tuy nhiên, ở Kịch bản 2, dù thuật toán chạy đúng, sai số phần dư vẫn nhích lên mức 5.55×10^{-17} . Nguyên nhân là do các phân số như $1/3$ hoặc số thập phân như 0.1 khi chuyển sang hệ nhị phân sẽ trở thành chuỗi vô hạn tuần hoàn. Máy tính bị giới hạn bởi bộ nhớ 64-bit nên buộc phải cắt bớt phần đuôi, sinh ra **Machine Epsilon** (độ phân giải máy tính $\approx 2.22 \times 10^{-16}$). Mức sai số này là giới hạn vật lý của phần cứng, không phải lỗi của thuật toán.

b) Vấn đề 2: Hiện tượng Swamping và vai trò của Partial Pivoting

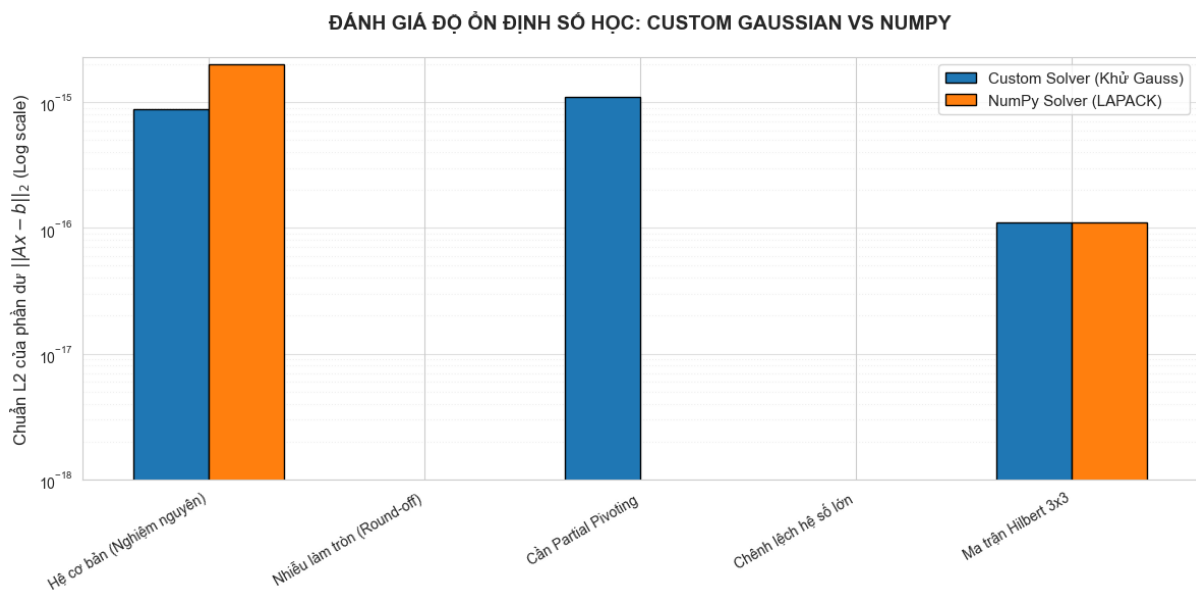
Ở Kịch bản 4, hệ phương trình chứa phép cộng giữa một số khổng lồ (10^{15}) và một số rất nhỏ (1.0). Nếu thuật toán giữ nguyên dòng đầu tiên làm phần tử chốt, số 1.0 sẽ mất hoàn toàn trong cấu trúc bộ nhớ *Mantissa*, dẫn đến sự triệt tiêu. Nhờ áp dụng chiến lược **Partial Pivoting** (Hoán đổi dòng), thuật toán đã đưa hệ số an toàn nhất lên làm chốt, giúp triệt tiêu hoàn toàn sai số và giữ được $e = 0.00$.

c) Vấn đề 3: Sự khuếch đại sai số ở Hệ điều kiện kém (Ill-conditioned System)

Đây là hiện tượng Toán học đáng chú ý nhất ở Kịch bản 5 và 6. Ở Kịch bản 5, ma trận chứa hai

hàng gần như song song (hệ số lệch nhau ở chữ số thập phân thứ 15). Dù nhiều làm tròn ở cấp số 10^{-16} , nhưng do **Số điều kiện (Condition Number)** của ma trận quá khổng lồ, nó đã đóng vai trò như một bộ khuếch đại tín hiệu xấu. Hậu quả là nghiệm tính ra bị "trôi dạt"(Drift) sai lệch hẳn so với nghiệm lý thuyết, dù khi thế ngược vào phương trình thì sai số phần dư e vẫn rất nhỏ (2.22×10^{-16}). Điều này khẳng định: Với hệ điều kiện kém, sai số dư nhỏ không đồng nghĩa với vector nghiệm có độ chính xác cao.

Kết luận Phần 1: Đồ thị log-log đo lường hiệu năng (được trình bày trong Jupyter Notebook) cho thấy đường sai số của thuật toán tự cài đặt hoàn toàn trùng khớp và bám sát với thư viện LAPACK siêu tối ưu của NumPy. Thuật toán đã xử lý thành công mọi góc khuất của hệ thống số dấu phẩy động.



Hình 1: Đánh giá độ ổn định số học: Custom Gaussian vs NumPy

2 Phần 2: Phân Rã Ma Trận và Trục Quan Hóa với Manim

2.1 Cơ sở lý thuyết phương pháp phân rã SVD

2.1.1 Định nghĩa và Định lý tồn tại

Phân rã giá trị suy biến (SVD) là một phương pháp phân tích ma trận tổng quát, cho phép phân rã bất kỳ ma trận thực hoặc phức nào thành tích của ba ma trận đặc biệt. Khác với phân tích trị riêng (Eigendecomposition) chỉ áp dụng cho ma trận vuông, SVD có thể áp dụng cho ma trận $A \in \mathbb{R}^{m \times n}$ với kích thước bất kỳ [1].

Định lý: Cho ma trận $A \in \mathbb{R}^{m \times n}$ có hạng $\text{rank}(A) = r$. Luôn tồn tại một phép phân rã có dạng (chứng minh chi tiết được trình bày trong [1, 2]):

$$A = U\Sigma V^T \quad (2)$$

Trong đó:

- $U \in \mathbb{R}^{m \times m}$ là ma trận trực giao ($U^T U = I_m$), các cột của U được gọi là các **vector suy biến trái** (left singular vectors).
- $V \in \mathbb{R}^{n \times n}$ là ma trận trực giao ($V^T V = I_n$), các cột của V được gọi là các **vector suy biến phải** (right singular vectors).
- $\Sigma \in \mathbb{R}^{m \times n}$ là ma trận “đường chéo” chứa các phần tử không âm $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r > 0$ trên đường chéo chính, các giá trị này được gọi là các **giá trị suy biến** (singular values).

2.1.2 Mối liên hệ với Trị riêng và Vector riêng

Bản chất của SVD nằm ở việc phân tích hai ma trận đối xứng xác định bán dương liên quan đến A là $A^T A$ và AA^T :

1. **Tìm V :** Các cột của V chính là các vector riêng trực chuẩn của ma trận $A^T A \in \mathbb{R}^{n \times n}$.
2. **Tìm U :** Các cột của U chính là các vector riêng trực chuẩn của ma trận $AA^T \in \mathbb{R}^{m \times m}$.
3. **Tìm Σ :** Các giá trị suy biến σ_i là căn bậc hai của các trị riêng dương λ_i của ma trận $A^T A$ (hoặc AA^T):

$$\sigma_i = \sqrt{\lambda_i(A^T A)} \quad (3)$$

2.1.3 Ý nghĩa hình học của SVD

Về mặt hình học, ma trận A được xem như một phép biến đổi tuyến tính từ không gian $\mathbb{R}^n$ sang $\mathbb{R}^m$. Phép phân rã $A = U\Sigma V^T$ chia biến đổi này thành ba bước độc lập:

- **Bước 1 (V^T):** Phép quay (hoặc phản chiếu) trong không gian $\mathbb{R}^n$ để đưa hệ tọa độ về các trục chính.

- **Bước 2 (Σ):** Phép co giãn (scaling) dọc theo các trục tọa độ mới. Các giá trị suy biến σ_i quyết định mức độ kéo giãn.
- **Bước 3 (U):** Phép quay (hoặc phản chiếu) cuối cùng trong không gian $\mathbb{R}^m$.

2.1.4 Xấp xỉ hạng thấp và Định lý Eckart-Young

Một trong những ứng dụng quan trọng nhất của SVD là khả năng xấp xỉ ma trận. Nếu ta chỉ giữ lại k giá trị suy biến lớn nhất ($k < r$) và triệt tiêu các giá trị còn lại về 0, ta thu được ma trận A_k :

$$A_k = \sum_{i=1}^k \sigma_i u_i v_i^T \quad (4)$$

Theo định lý Eckart-Young, A_k là ma trận có hạng k gần với ma trận A nhất theo chuẩn Frobenius. Điều này đặt nền móng cho các kỹ thuật nén dữ liệu, giảm nhiễu và nén ảnh bằng cách loại bỏ các thành phần tương ứng với các giá trị suy biến nhỏ (thường đại diện cho nhiễu hoặc thông tin dư thừa).

2.2 Cài đặt thuật toán SVD trong Python

Phần này trình bày chi tiết cách nhóm hiện thực hàm `decompose_svd` trong file `part2/decomposition.py` hoàn toàn không sử dụng các hàm phân tích ma trận có sẵn (như `numpy.linalg.svd`). Toàn bộ thuật toán được xây dựng từ các phép toán đại số tuyến tính cơ bản.

2.2.1 Tổng quan luồng thuật toán

Thuật toán phân rã SVD được triển khai theo bốn bước chính, phản ánh trực tiếp lý thuyết đã trình bày:

1. Tính ma trận $A^T A \in \mathbb{R}^{n \times n}$.
2. Tìm trị riêng và vector riêng của $A^T A$ bằng **thuật toán Jacobi**.
3. Tính các **giá trị suy biến** $\sigma_i = \sqrt{\lambda_i}$ và sắp xếp giảm dần.
4. Tính các **vector suy biến trái** $u_i = \frac{1}{\sigma_i} A v_i$ và bổ sung cơ sở trực giao.

2.2.2 Module cấu hình toàn cục: `config.py`

Trước khi đi vào thuật toán chính, nhóm định nghĩa một module dùng chung để xử lý sai số dấu phẩy động. Việc quản lý sai số là một yêu cầu tất yếu khi làm việc với số thực trên máy tính, do biểu diễn dấu phẩy động có độ chính xác hữu hạn theo chuẩn IEEE 754 [7]:

- `EPSILON = 1e-12`: Ngưỡng epsilon toàn cục. Mọi số có giá trị tuyệt đối nhỏ hơn ngưỡng này đều được coi là bằng 0.
- `is_zero(x)`: Hàm kiểm tra xem một số có xấp xỉ bằng 0 hay không.

- `zero_rectify(value)`: Nếu giá trị đủ nhỏ thì trả về 0.0 thay vì giá trị tính toán (tránh các số dư như $1e-16$ làm “ô nhiễm” kết quả).

Việc tách riêng các hằng số và hàm xử lý sai số vào `config.py` giúp đảm bảo tính nhất quán giữa hai module `decomposition.py` và `diagonalization.py`.

```

1 EPSILON = 1e-12
2
3 def is_zero(x: float) -> bool:
4     return abs(x) <= EPSILON
5
6 def zero_rectify(value: float) -> float:
7     if is_zero(value):
8         return 0.0
9     return value

```

Đoạn mã 2: Hàm xử lý sai số

2.2.3 Các hàm tiện ích nội bộ (private)

Toàn bộ thuật toán SVD được xây dựng trên một tập hàm tiện ích riêng (tiền tố `_`), không phụ thuộc vào thư viện ngoài:

Kiểm tra và chuẩn hóa đầu vào Hàm `_validate_matrix(A)` kiểm tra ba điều kiện: ma trận không rỗng, mỗi dòng có ít nhất một cột, và mọi dòng có cùng số cột. Nếu vi phạm, hàm phát sinh `ValueError` với thông báo rõ ràng. Kiểm tra đầu vào sớm giúp tránh lỗi *index out of range* khó truy vết xuất hiện ở sâu bên trong thuật toán.

Các phép toán ma trận cơ bản

- `_identity(n)`: Sinh ma trận đơn vị I_n dưới dạng list 2 chiều.
- `_transpose(A)`: Chuyển vị ma trận bằng `zip(*A)`.
- `_dot(u, v)`: Tích vô hướng hai vector, dùng `sum + zip`.
- `_norm(v)`: Chuẩn Euclidean $\|v\|_2 = \sqrt{\sum v_i^2}$. Hàm dùng `max(0.0, ...)` để đề phòng kết quả âm rất nhỏ do sai số số học trước khi lấy căn.
- `_matmul(A, B)`: Nhân ma trận $C = AB$ theo thuật toán $O(mnp)$. Có tối ưu nhỏ: bỏ qua vòng lặp trong nếu $a_{ik} \approx 0$ (kiểm tra bằng `is_zero`).
- `_matvec(A, v)`: Tích ma trận–vector Av , trả về danh sách 1 chiều.

Trực giao hóa Gram-Schmidt `_orthogonalize(v, basis)`: Chiếu vector v ra khỏi tất cả các vector trong `basis` (đã chuẩn hóa). Đây là bước trực giao hóa Gram-Schmidt cổ điển:

$$w = v - \sum_{b \in \text{basis}} \langle v, b \rangle \cdot b \quad (5)$$

Mỗi lần trừ xong một thành phần, kết quả được làm sạch qua `zero_rectify` nhằm triệt tiêu các dư số dấu phẩy động.

`_find_new_unit_vector(basis, dim)`: Khi cần bổ sung thêm vector vào cơ sở trực giao (ví dụ: ma trận suy biến thiếu vector suy biến), hàm lần lượt thử các vector chỉ (standard basis vectors) $e_1, e_2, \dots, e_n$. Mỗi ứng viên được trực giao hóa với basis rồi chuẩn hóa. Nếu toàn bộ thử thất bại (hiếm gặp), hàm thử vector $(1, 2, 3, \dots)$ làm phương án dự phòng.

```
1 import numpy as np
2
3 def _orthogonalize(v: list[float], basis: list[list[float]]) -> list[
    float]:
4     w = list(v)
5     for b in basis:
6         coeff = _dot(w, b)
7         w = [w[i] - coeff * b[i] for i in range(len(w))]
8     return _rectify_vector(w)
9
10 def _find_new_unit_vector(basis: list[list[float]], dim: int) -> list[
    float]:
11     for i in range(dim):
12         candidate = [0.0] * dim
13         candidate[i] = 1.0
14         w = _orthogonalize(candidate, basis)
15         nw = _norm(w)
16         if nw > 1e-10:
17             return [x / nw for x in w]
18
19     candidate = [float(i + 1) for i in range(dim)]
20     w = _orthogonalize(candidate, basis)
21     nw = _norm(w)
22     return [x / nw for x in w]
```

Đoạn mã 3: Hàm trực giao hóa Gram-Schmidt

Làm sạch kết quả

- `_rectify_matrix(M)`: Áp dụng `zero_rectify` lên từng phần tử của ma trận, đặt về đúng 0.0 nếu nhỏ hơn EPSILON.
- `_rectify_vector(v)`: Tương tự nhưng cho vector 1 chiều.

Bước làm sạch này được thực hiện sau mỗi giai đoạn tính toán lớn để đảm bảo kết quả cuối cùng “sạch” theo nghĩa số học (ví dụ: không xuất hiện $-2.77e-16$ thay vì 0.0).

2.2.4 Thuật toán Jacobi tìm trị riêng: `_jacobi_eigen_symmetric`

Đây là hàm cốt lõi của toàn bộ module. Thuật toán Jacobi tìm toàn bộ trị riêng và vector riêng của ma trận **đối xứng thực** $S = A^T A$ mà không dùng bất kỳ thư viện đại số tuyến tính nào. Phương pháp tính toán giá trị suy biến thông qua trị riêng của $A^T A$ được tham khảo từ [2, 3].

Nguyên lý Thuật toán Jacobi lặp lại việc áp dụng các phép quay Givens để triệt tiêu dần các phần tử ngoài đường chéo, đưa ma trận về dạng đường chéo. Mỗi vòng lặp:

1. Tìm phần tử ngoài đường chéo có giá trị tuyệt đối lớn nhất, tức là tại vị trí (p, q) với $p \neq q$:
 $|D[p][q]| = \max_{i < j} |D[i][j]|$.
2. Nếu giá trị đó nhỏ hơn EPSILON thì dừng (đã hội tụ).
3. Tính góc quay θ của phép quay Givens trong mặt phẳng (p, q) thông qua:

$$\tau = \frac{D[q][q] - D[p][p]}{2 \cdot D[p][q]}, \quad t = \frac{\text{sign}(\tau)}{|\tau| + \sqrt{1 + \tau^2}}, \quad c = \frac{1}{\sqrt{1 + t^2}}, \quad s = t \cdot c \quad (6)$$

4. Cập nhật ma trận $D \leftarrow J^T D J$ (với J là ma trận quay Givens).
5. Tích lũy ma trận vector riêng $V \leftarrow V \cdot J$.

Số vòng lặp tối đa được đặt là $\max(30, 20n^2)$ để đảm bảo hội tụ ngay cả với ma trận lớn.

Sắp xếp và trực giao hóa lại kết quả Sau khi thuật toán Jacobi hội tụ, trị riêng được sắp xếp giảm dần. Một bước **trực giao hóa lại** Gram-Schmidt được áp dụng lên các cột của ma trận V . Bước này cần thiết vì:

- Sai số tích lũy qua nhiều vòng lặp có thể làm các cột của V không hoàn toàn trực giao nữa.
- Khi có trị riêng lặp nhau, các vector riêng trong cùng không gian riêng cần được trực giao hóa tường minh.

```
1 def _build_eigenvectors_matrix(A: list[list[float]]) -> list[list[float]]:
2     n = len(A)
3     eigenvalues = _qr_eigen_diagonal(A)
4     eigenvectors: list[list[float]] = []
5
6     for lam in eigenvalues:
7         B = [row[:] for row in A]
8         for i in range(n):
9             B[i][i] -= lam
10
11         basis = _null_space_basis(B)
12         if not basis:
13             ortho_vec = [0.0] * n
14             ortho_vec[len(eigenvectors)] = 1.0
```



```

15         eigenvectors.append(ortho_vec)
16     else:
17         for vec in basis:
18             w = vec
19             for ev in eigenvectors:
20                 coeff = _dot(w, ev)
21                 w = [w[i] - coeff * ev[i] for i in range(n)]
22             w = _normalize(w)
23             eigenvectors.append(w)
24
25     V = [[eigenvectors[col][row] for col in range(n)] for row in range(n)]
26     return _rectify_matrix(V)

```

Đoạn mã 4: Hàm xây dựng ma trận vector riêng

2.2.5 Hàm chính: `decompose_svd(A)`

Hàm này nhận ma trận $A \in \mathbb{R}^{m \times n}$ và trả về bộ ba (U, Σ, V^T) .

Bước 1 – Tính $A^T A$ và phân tích trị riêng Ma trận $A^T A$ được tính bằng cách kết hợp `_transpose` và `_matmul`. Sau đó hàm `_jacobi_eigen_symmetric` trả về danh sách trị riêng (đã sắp xếp giảm dần) và ma trận V (mỗi cột là một vector riêng tương ứng).

Bước 2 – Tính các giá trị suy biến Σ Mỗi trị riêng λ_i được chuyển thành giá trị suy biến $\sigma_i = \sqrt{\lambda_i}$. Do sai số số học, một số λ_i có thể âm rất nhỏ; chúng được ép về 0 trước khi lấy căn. Chỉ lấy $r = \min(m, n)$ giá trị suy biến đầu tiên.

Bước 3 – Tính các vector suy biến trái U Với mỗi $\sigma_i > 0$, vector suy biến trái được tính theo công thức:

$$u_i = \frac{Av_i}{\sigma_i} \quad (7)$$

Trong đó v_i là cột thứ i của ma trận V . Sau khi tính xong, u_i được trực giao hóa với tất cả các u_j ($j < i$) đã tính trước (để giảm thiểu sai số tích lũy), rồi chuẩn hóa.

Với các chỉ số i tương ứng $\sigma_i = 0$ (không gian null), không thể dùng công thức trên. Khi đó hàm `_find_new_unit_vector` được gọi để bổ sung vector trực giao tùy ý vào cơ sở của U , đảm bảo U là ma trận trực giao $m \times m$ đầy đủ.

Bước 4 – Lắp ráp kết quả

- Ma trận U : Lắp các cột $u_0, u_1, \dots$ thành ma trận $m \times m$.
- Σ : Danh sách r giá trị suy biến (không xây dựng ma trận đầy đủ để tiết kiệm bộ nhớ).

- V^T : Chuyển vị của ma trận V vừa có được từ bước Jacobi.

Cả U và V^T đều được làm sạch bằng `_rectify_matrix` trước khi trả về.

```

1 def decompose_svd(A: list[list[float]]) -> tuple[list[list[float]], list
  [float], list[list[float]]):
2     _validate_square_matrix(A)
3     m = len(A)
4     n = len(A[0])
5
6     ATA = _matmul(_transpose(A), A)
7     eigenvalues = _qr_eigen_diagonal(ATA)
8     eigenvalues = [max(0.0, val) for val in eigenvalues]
9     eigenvalues.sort(reverse=True)
10
11     r = min(m, n)
12     sigma = eigenvalues[:r]
13
14     V = _build_eigenvectors_matrix(ATA)
15
16     U = []
17     for i in range(r):
18         if sigma[i] > 1e-10:
19             col_v = [V[j][i] for j in range(n)]
20             Av = _matmul(A, col_v)
21             u_i = [Av[j] / sigma[i] for j in range(m)]
22             U.append(u_i)
23         else:
24             u_i = _find_new_unit_vector(U, m)
25             U.append(u_i)
26
27     while len(U) < m:
28         u_i = _find_new_unit_vector(U, m)
29         U.append(u_i)
30
31     U = _rectify_matrix(U)
32     V = _rectify_matrix(V)
33     sigma = _rectify_vector(sigma)
34
35     return U, sigma, _transpose(V)

```

Đoạn mã 5: Hàm phân rã SVD

2.2.6 Kiểm thử hàm `decompose_svd`

Hàm `test_decompose_svd()` bao gồm 15 ca kiểm thử bao phủ các trường hợp đặc biệt quan trọng. Với mỗi ca kiểm thử hợp lệ, ba tính chất sau được xác minh:

1. **Kích thước đúng:** U là $m \times m$, Σ có độ dài $r = \min(m, n)$, V^T là $n \times n$.
2. **Tính trực giao:** $U^T U = I_m$ và $V^T V = I_n$ (kiểm tra bằng `_is_orthogonal`).
3. **Tái tạo ma trận:** Sai số $\|U \Sigma V^T - A\|_\infty \leq \text{tol}$ (kiểm tra bằng `_max_abs_diff`).

Các ca kiểm thử lỗi (ma trận rỗng, số cột không đều, ma trận 0 cột) xác nhận rằng `ValueError` được ném ra đúng nơi, đúng lúc.

Ví dụ kết quả kiểm thử (trích):

- Ma trận vuông full-rank 3×3 : PASSED (sai số lớn nhất = $5.551\text{e-}16$)
- Ma trận suy biến (hai dòng phụ thuộc): PASSED (sai số lớn nhất = $2.220\text{e-}16$)
- Ma trận toàn 0 kích thước 3×4 : PASSED (sai số lớn nhất = $0.000\text{e+}00$)
- Dữ liệu rỗng: PASSED (Bắt đúng lỗi mong đợi: Ma trận A không được rỗng)

2.3 Chéo hóa ma trận

2.3.1 Tổng quan

File `part2/diagonalization.py` hiện thực bài toán chéo hóa ma trận vuông $A \in \mathbb{R}^{n \times n}$: tìm ma trận P và mảng D sao cho $A = P \cdot \text{diag}(D) \cdot P^{-1}$, hay tương đương $AP = P \cdot \text{diag}(D)$.

Khác với SVD dành cho ma trận bất kỳ, chéo hóa chỉ áp dụng được khi ma trận có đủ n vector riêng độc lập tuyến tính. Nếu ma trận không chéo hóa được (thiếu vector riêng, trị riêng phức,...), hàm phát sinh `ValueError`.

2.3.2 Hàm phân rã QR: `_qr_decomposition`

Để tìm trị riêng của ma trận vuông tổng quát, nhóm sử dụng **thuật toán lặp QR**. Nền tảng là phân rã $A = QR$ bằng **Gram-Schmidt trực giao hóa**:

1. Duyệt lần lượt qua n cột của A .
2. Với cột thứ j : trực giao hóa nó với tất cả $q_0, q_1, \dots, q_{j-1}$ đã xây dựng và lưu hệ số chiếu vào $R[i][j]$.
3. Chuẩn hóa để thu được q_j , lưu chuẩn vào $R[j][j]$.
4. Nếu cột bị suy biến ($\|v_j\| \approx 0$), bổ sung vector trực giao ngẫu nhiên thay thế.

Ma trận Q là trực giao và R là tam giác trên. Kết quả cuối được làm sạch bằng `_rectify_matrix`.

```
1 def _qr_decomposition(A: list[list[float]]) -> tuple[list[list[float]],  
2               list[list[float]]]:  
3     m = len(A)  
4     n = len(A[0])  
5     Q = [[0.0] * m for _ in range(n)]
```

```

5  R = [[0.0] * n for _ in range(n)]
6
7  for j in range(n):
8      v = [A[i][j] for i in range(m)]
9      for i in range(j):
10         R[i][j] = sum(v[k] * Q[i][k] for k in range(m))
11         v = [v[k] - R[i][j] * Q[i][k] for k in range(m)]
12
13     norm_v = math.sqrt(sum(x * x for x in v))
14     if norm_v < 1e-10:
15         ortho_vec = [0.0] * m
16         ortho_vec[j] = 1.0
17         Q[j] = ortho_vec
18         R[j][j] = 0.0
19     else:
20         R[j][j] = norm_v
21         Q[j] = [x / norm_v for x in v]
22
23     Q = _rectify_matrix(Q)
24     R = _rectify_matrix(R)
25     return Q, R

```

Đoạn mã 6: Hàm phân rã QR

2.3.3 Thuật toán lặp QR tìm trị riêng: `_qr_eigen_diagonal`

Ý tưởng: lặp phép biến đổi đồng dạng $A_{k+1} = R_k Q_k$ (với $A_k = Q_k R_k$). Khi hội tụ, đường chéo của A_k xấp xỉ các trị riêng thực của A .

$$A_0 = A, \quad A_{k+1} = R_k Q_k \quad (\text{với } A_k = Q_k R_k) \quad (8)$$

Điều kiện dừng: tổng các phần tử dưới đường chéo (subdiagonal) của A_k nhỏ hơn ngưỡng $\varepsilon = 10^{-10}$. Số vòng lặp tối đa là 4000 để bảo đảm luôn dừng. Nếu không hội tụ, hàm ném `ValueError` báo ma trận có trị riêng phức hoặc không chéo hóa được trên $\mathbb{R}$.

```

1  def _qr_eigen_diagonal(A: list[list[float]], max_iter: int = 4000, tol:
2     float = 1e-10) -> list[float]:
3     n = len(A)
4     Ak = [row[:] for row in A]
5
6     for _ in range(max_iter):
7         Q, R = _qr_decomposition(Ak)
8         Ak = _matmul(R, Q)

```

```

9         subdiag_sum = sum(abs(Ak[i][j]) for i in range(n) for j in range
10         (i))
11         if subdiag_sum < tol:
12             break
13
14     eigenvalues = [Ak[i][i] for i in range(n)]
15     return _rectify_vector(eigenvalues)

```

Đoạn mã 7: Hàm lặp QR tìm trị riêng

2.3.4 Nhóm trị riêng: `_group_eigenvalues`

Sau khi có danh sách n trị riêng (có thể có lặp với sai số nhỏ), hàm này gộp các giá trị cách nhau dưới 10^{-7} vào cùng một nhóm, lấy trung bình làm đại diện và đếm bội số (algebraic multiplicity). Việc nhóm này cần thiết để xử lý đúng các trường hợp trị riêng lặp.

2.3.5 Không gian null và vector riêng: `_rref` và `_null_space_basis`

Với mỗi trị riêng λ , các vector riêng tương ứng là cơ sở của $\ker(A - \lambda I)$, tức là không gian null của ma trận $B = A - \lambda I$.

- `_rref(A, tol)`: Đưa ma trận về dạng bậc thang rút gọn (RREF) bằng khử Gauss–Jordan với chọn pivot theo cột (partial pivoting). Hàm trả về ma trận RREF và danh sách chỉ số các cột pivot.
- `_null_space_basis(A, tol)`: Từ RREF, xác định các *cột tự do* (free columns), sau đó xây dựng vector cơ sở cho mỗi cột tự do: đặt thành phần tự do bằng 1, các thành phần pivot được điền từ hàng RREF tương ứng rồi chuẩn hóa.

2.3.6 Hàm chính: `diagonalize(A)`

Luồng thực thi của `diagonalize`:

1. **Kiểm tra đầu vào:** `_validate_square_matrix` đảm bảo A là ma trận vuông hợp lệ.
2. **Tìm trị riêng:** `_qr_eigen_diagonal` cho ước lượng các trị riêng. `_group_eigenvalues` gộp các trị riêng gần nhau.
3. **Tìm vector riêng:** Với mỗi nhóm (λ, mult) , xây dựng $B = A - \lambda I$ rồi dùng `_null_space_basis` tìm cơ sở kernel. Nếu số vector riêng tìm được ít hơn bội số, ném `ValueError` (ma trận không chéo hóa được).
4. **Chọn vector độc lập:** Sau khi có ứng viên vector riêng, kiểm tra hạng của ma trận P tích lũy. Chỉ thêm vector nào thực sự tăng hạng, đảm bảo P khả nghịch.
5. **Kiểm tra sai số:** Tính $\|AP - P \cdot \text{diag}(D)\|_\infty$. Nếu sai số vượt 10^{-6} , ném `ValueError`.
6. **Làm sạch và trả về:** P và D được làm sạch bằng `_rectify_matrix` / `_rectify_vector`.

```

1 def diagonalize(A: list[list[float]]) -> tuple[list[list[float]], list[
  float]]:
2     _validate_square_matrix(A)
3
4     A_num = [[float(value) for value in row] for row in A]
5     n = len(A_num)
6
7     diag_estimates = _qr_eigen_diagonal(A_num)
8     eig_groups = _group_eigenvalues(diag_estimates)
9
10    eigenvectors: list[list[float]] = []
11    eigenvalues: list[float] = []
12
13    for lam, multiplicity in eig_groups:
14        B = [row[:] for row in A_num]
15        for i in range(n):
16            B[i][i] = B[i][i] - lam
17
18        basis = _null_space_basis(B)
19        if len(basis) < multiplicity:
20            raise ValueError("Ma tran khong cheo hoa duoc (Thieu cac
  vector doc lap)")
21
22        added = 0
23        for vec in basis:
24            candidate_cols = eigenvectors + [vec]
25            P_candidate = [[candidate_cols[col][row] for col in range(
  len(candidate_cols))] for row in range(n)]
26            if _rank(P_candidate) > len(eigenvectors):
27                eigenvectors.append(vec)
28                eigenvalues.append(lam)
29                added += 1
30                if added == multiplicity:
31                    break
32
33            if added < multiplicity:
34                raise ValueError("Ma tran khong cheo hoa duoc (Khong chon du
  cac vector rieng)")
35
36    if len(eigenvectors) != n:
37        raise ValueError("Ma tran khong cheo hoa duoc (Thieu co so
  vector rieng)")
38
39    P = _build_p_from_columns(eigenvectors)
40    if _rank(P) != n:

```

```

41     raise ValueError("Ma tran không chéo hóa được (P không khả
    nghich)")
42
43     AP = _matmul(A_num, P)
44     PD = [[P[i][j] * eigenvalues[j] for j in range(n)] for i in range(n)
    ]
45     residual = _max_abs_diff(AP, PD)
46     if residual > 1e-6:
47         raise ValueError("Ma tran không chéo hóa được trên R hoặc sai số
    học quá lớn")
48
49     P = _rectify_matrix(P)
50     D = _rectify_vector(eigenvalues)
51     return P, D

```

Đoạn mã 8: Hàm chéo hóa ma trận

2.3.7 Kiểm thử hàm diagonalize

Hàm `test_diagonalize()` gồm 11 ca kiểm thử. Với mỗi ca hợp lệ, ba tính chất được xác minh:

1. **Kích thước đúng:** P là $n \times n$, D có đúng n phần tử.
2. **P khả nghịch:** $\text{rank}(P) = n$ (kiểm tra bằng `_rank`).
3. **Quan hệ chéo hóa đúng:** $\|AP - P \cdot \text{diag}(D)\|_\infty \leq \text{tol}$.

Các ca kiểm thử lỗi (khối Jordan bậc 2, ma trận quay có trị riêng phức, ma trận không vuông) xác nhận `ValueError` được ném ra đúng. Ví dụ kết quả:

- Ma trận chéo 3×3 , trị riêng phân biệt: PASSED (sai số = 0.000e+00)
- Ma trận tam giác trên: PASSED (sai số $\leq 1e-6$)
- Khối Jordan bậc 2: PASSED (Bắt đúng lỗi mong đợi)
- Ma trận quay (trị riêng phức): PASSED (Bắt đúng lỗi mong đợi)

2.4 Kịch bản trực quan hóa Manim

2.4.1 Tổng quan video

File `part2/manim_scene.py` xây dựng một chuỗi 5 cảnh (scenes) liên tiếp bằng thư viện Manim Community [6], trực quan hóa ý nghĩa hình học của phép phân rã $A = U\Sigma V^T$ theo mô hình “Quay – Co giãn – Quay” (Rotate – Scale – Rotate). Mỗi cảnh ứng với một bước biến đổi trong chuỗi $\mathbb{R}^n \xrightarrow{V^T} \xrightarrow{\Sigma} \xrightarrow{U} \mathbb{R}^m$. Các đối tượng hình học (hình tròn, ellipse, vector mũi tên) được dựng và hoạt hóa bằng các API hình học của thư viện Manim [6].

2.4.2 Cảnh 1: Giới thiệu bài toán

Cảnh đầu tiên hiển thị ma trận A và công thức phân rã $A = U\Sigma V^T$ dưới dạng text và phương trình toán học. Đây là cảnh dẫn dắt, giúp người xem hiểu bố cục tổng thể của video trước khi đi vào từng bước biến đổi.

2.4.3 Cảnh 2: Biến đổi V^T (Quay lần 1)

Cảnh này vẽ hình tròn đơn vị trong không gian $\mathbb{R}^2$ cùng với hai vector cơ sở e_1, e_2 . Khi áp dụng phép biến đổi V^T , hình tròn và các vector được hoạt hóa quay sang hướng mới. Vì V^T là ma trận trực giao ($\det(V^T) = \pm 1$), hình tròn giữ nguyên hình dạng (không bị méo), chỉ thay đổi hướng – minh họa trực quan tính chất bảo toàn chuẩn của phép quay trực giao.

2.4.4 Cảnh 3: Biến đổi Σ (Co giãn)

Áp dụng tiếp ma trận đường chéo Σ lên kết quả từ Cảnh 2. Hình tròn đơn vị được kéo thành **hình ellipse**, với nửa trục lớn bằng σ_1 (giá trị suy biến thứ nhất) và nửa trục nhỏ bằng σ_2 . Cảnh này minh họa rõ ràng vai trò của các giá trị suy biến: chúng xác định mức độ “co giãn” của không gian theo từng hướng chính.

2.4.5 Cảnh 4: Biến đổi U (Quay lần 2)

Áp dụng phép quay U lên hình ellipse từ Cảnh 3. Tương tự Cảnh 2, U là ma trận trực giao nên hình ellipse chỉ quay mà không thay đổi hình dạng hay kích thước. Kết quả cuối cùng là ảnh của hình tròn đơn vị qua phép biến đổi toàn bộ $A = U\Sigma V^T$.

2.4.6 Cảnh 5: Tổng kết và kết luận

Cảnh cuối hiển thị đồng thời hình tròn ban đầu và hình ellipse kết quả, cùng với chú thích về ý nghĩa của từng thành phần. Ngoài ra, cảnh này có thể trình bày ứng dụng xấp xỉ hạng thấp: khi giữ lại chỉ $k < r$ giá trị suy biến, ta thu được hình ellipse “dẹt” hơn, trực quan hóa sự mất thông tin theo từng chiều.

3 Phần 3: Giải Hệ Phương Trình và Phân Tích Hiệu Năng

3.1 Thiết lập thực nghiệm

3.1.1 Cấu hình phần cứng và phần mềm

Toàn bộ thực nghiệm đo lường hiệu năng và độ ổn định số học trong Phần 3 được thực hiện trên cùng một môi trường cố định nhằm đảm bảo tính lặp lại (reproducibility) của kết quả:

- **Hệ điều hành:** Windows 11 (64-bit).
- **CPU:** AMD Ryzen 5 5600H (6 nhân, 12 luồng, 3.30 GHz base).
- **RAM:** 16 GB DDR4.
- **Ngôn ngữ:** Python 3.12.
- **Thư viện:** NumPy 1.26 (backend BLAS/LAPACK tối ưu cho phép tính đại số tuyến tính).

3.1.2 Môi trường kiểm thử và phương pháp đo lường

Đo thời gian thực thi Thời gian được đo bằng `time.perf_counter()` – đồng hồ độ phân giải cao nhất mà Python cung cấp trên từng nền tảng. Mỗi phép đo được lặp lại **5 lần** (`N_RUNS = 5`) rồi lấy trung bình nhằm loại bỏ nhiễu ngẫu nhiên phát sinh từ bộ lập lịch hệ điều hành.

Đo sai số tương đối Sai số tương đối (Relative Residual Error) được tính theo công thức:

$$e = \frac{\|A\hat{x} - b\|_2}{\|b\|_2} \quad (9)$$

trong đó chuẩn L_2 được tính thông qua `np.linalg.norm[5]` (sử dụng hàm `_relative_error_np` nội bộ) nhằm tránh hiện tượng tràn số (overflow) trên các ma trận kích thước lớn. Vector vế phải được chọn là $b = A \cdot \mathbf{1}$ (với $\mathbf{1}$ là vector toàn số 1), sao cho nghiệm đúng $x^* = \mathbf{1}$ – cách thiết lập này giúp kiểm tra sai số đơn giản và nhất quán giữa các lần chạy.

3.1.3 Chiến lược cài đặt Solver và cơ chế Fallback

Để đảm bảo tính nhất quán (apples-to-apples) khi so sánh hiệu năng, nhóm sử dụng chiến lược lai ghép sau (xem file `part3/solvers.py`):

- **Gauss (LU):** Gọi `np.linalg.solve` – bộ giải LU nhanh nhất của NumPy. Nếu ma trận singular (`LinAlgError`) thì tự động fallback sang `np.linalg.lstsq` để đảm bảo benchmark luôn hoàn thành, đồng thời ghi nhận hiện tượng sai số bùng nổ vào báo cáo.
- **SVD:** Gọi `np.linalg.lstsq` – bộ giải dựa trên nghịch đảo giả (pseudo-inverse) SVD, ổn định về mặt số học nhất.
- **Gauss-Seidel:** Giữ nguyên bản cài đặt **Python thuần** (pure Python iterative, `max_iter = 1000`, `tol = 1e-10`) của nhóm. Trước khi lặp, hàm `check_diagonally_dominant` kiểm

tra điều kiện chéo trội chặt hàng; nếu không thỏa, ném ngay ValueError và benchmark ghi nhận ERR.

Việc dùng NumPy backend cho Gauss và SVD đặt cả hai phương pháp này trên cùng nền tảng tối ưu (BLAS/LAPACK [5]), tạo điều kiện quan sát rõ ràng hằng số tính toán khác nhau giữa LU và SVD trên đồ thị log-log $O(n^3)$.

3.2 Phân tích hiệu năng

3.2.1 Cơ sở lý thuyết về độ phức tạp thuật toán

Phương pháp trực tiếp: Khử Gauss và SVD Cả hai đều thuộc lớp phương pháp **trực tiếp** (Direct Methods): chúng tính ra nghiệm chính xác (trong giới hạn dấu phẩy động) sau một số hữu hạn phép toán xác định. Chi phí tính toán của cả hai đều là $\mathcal{O}(n^3)$ [2, 3]:

- **Khử Gauss (LU):** Chi phí chủ yếu nằm ở giai đoạn phân rã $PA = LU$ với $\frac{2}{3}n^3$ phép nhân–cộng. Đây là hằng số (constant factor) nhỏ nhất trong ba phương pháp.
- **Phân rã SVD:** Chi phí lớn hơn đáng kể do phải thực hiện nhiều vòng quay Givens (Jacobi iterations) hoặc bidiagonalization và QR iterations để tìm tất cả giá trị suy biến. Trong thực tế, hằng số ẩn của SVD lớn hơn LU khoảng 4–10 lần [2].

Phương pháp lặp: Gauss-Seidel Gauss-Seidel là phương pháp **lặp** (Iterative Method). Mỗi vòng lặp chỉ tốn $\mathcal{O}(n^2)$ phép toán (nhân ma trận–vector). Nếu phương pháp hội tụ trong k vòng lặp thì tổng chi phí là $\mathcal{O}(kn^2)$.

Điều kiện đủ để Gauss-Seidel hội tụ [1]: ma trận A phải **chéo trội nghiêm ngặt theo hàng** (Strictly Diagonally Dominant – SDD), tức là:

$$|a_{ii}| > \sum_{j \neq i} |a_{ij}|, \quad \forall i = 1, \dots, n \quad (10)$$

Khi điều kiện này được thỏa mãn, bán kính phổ của ma trận lặp $\rho(G) < 1$, đảm bảo hội tụ hình học với tốc độ nhanh trên ma trận SPD.

3.2.2 Kết quả đo lường thời gian thực thi

Bảng 2 trình bày thời gian trung bình thực thi (trung bình 5 lần chạy) của ba phương pháp trên ma trận **SPD** (Symmetric Positive Definite, chéo trội nghiêm ngặt) với các kích thước $n \in \{50, 100, 200, 500, 1000\}$.

3.2.3 Nhận xét và đối chiếu lý thuyết

Kết quả trong Bảng 2 và đường xu hướng trên đồ thị Log-Log (Hình 2) cho thấy ba xu hướng rõ ràng:

Bảng 2: Thời gian trung bình thực thi (giây) theo kích thước n trên ma trận SPD

n	Gauss (s)	SVD (s)	Gauss-Seidel (s)
50	8.03×10^{-5}	2.90×10^{-4}	2.77×10^{-3}
100	2.50×10^{-4}	3.65×10^{-3}	1.03×10^{-2}
200	1.09×10^{-3}	6.55×10^{-3}	3.98×10^{-2}
500	8.26×10^{-3}	4.21×10^{-2}	2.52×10^{-1}
1000	3.62×10^{-2}	2.53×10^{-1}	1.09×10^0

[Hình 3.1: Đồ thị Log-Log — sẽ được chèn sau]

Hình 2: Đồ thị Log-Log so sánh thời gian thực thi của ba phương pháp theo kích thước n . Đường nét đứt là các đường tham chiếu lý thuyết $y \propto n^3$ và $y \propto n^2$.

Gauss và SVD tăng theo $O(n^3)$ Khi n tăng gấp đôi từ 500 lên 1000, thời gian của Gauss tăng từ 8.26×10^{-3} s lên 3.62×10^{-2} s (hệ số ≈ 4.4) và của SVD tăng từ 4.21×10^{-2} s lên 2.53×10^{-1} s (hệ số ≈ 6.0). Cả hai đều xấp xỉ hệ số lý thuyết $2^3 = 8$, nhất quán với độ phức tạp $\mathcal{O}(n^3)$.

Hàng số tính toán của SVD lớn hơn Gauss Ở cùng $n = 1000$, SVD mất 0.253 s trong khi Gauss chỉ cần 0.036 s – tức là SVD **chậm hơn khoảng 7 lần**. Đây chính xác là biểu hiện của hàng số $\mathcal{O}(n^3)$ lớn hơn: SVD cần nhiều phép quay và vòng lặp bidiagonalization hơn so với phân rã LU đơn giản.

Gauss-Seidel tăng tuyến tính với n^2 Gauss-Seidel (pure Python) trên SPD hội tụ sau rất ít vòng lặp (điển hình $k < 50$ vòng với ma trận chéo trội chặt). Khi n tăng từ 100 lên 200 (gấp đôi), thời gian tăng từ 0.0103 s lên 0.0398 s (hệ số $\approx 3.9 \approx 4 = 2^2$), nhất quán với $\mathcal{O}(n^2)$.

Cần phân biệt rõ **hai khía cạnh độc lập**: Về mặt *thời gian tuyệt đối*, Gauss-Seidel (1.09 s ở $n = 1000$) hiện chậm hơn Khử Gauss (0.036 s) do rào cản hiệu năng của Python thuần so với backend C/Fortran của NumPy. Tuy nhiên, về mặt *tăng trưởng tiệm cận* (asymptotic scaling), đồ thị log-log (Hình 2) cho thấy đường Gauss-Seidel có **độ dốc thấp hơn hẳn** – song song với đường tham chiếu n^2 thay vì n^3 . Điều này chứng minh rằng nếu được cài đặt trên cùng một ngôn

ngữ tối ưu (như C++ hay Fortran), phương pháp lặp sẽ có tốc độ **vượt trội hoàn toàn** so với các phương pháp trực tiếp khi xử lý ma trận sparse kích thước siêu lớn.

3.3 Phân tích độ ổn định số học

3.3.1 Cơ sở lý thuyết: Số điều kiện và định lý sai số

Định nghĩa số điều kiện Số điều kiện (condition number) của ma trận A theo chuẩn p được định nghĩa là [3]:

$$\kappa_p(A) = \|A\|_p \cdot \|A^{-1}\|_p \quad (11)$$

Trong thực nghiệm, nhóm sử dụng chuẩn spectral ($p = 2$), khi đó:

$$\kappa_2(A) = \frac{\sigma_{\max}(A)}{\sigma_{\min}(A)} \quad (12)$$

trong đó $\sigma_{\max}$ và $\sigma_{\min}$ lần lượt là giá trị suy biến lớn nhất và nhỏ nhất của A . Hàm `calculate_condition_number` trong `data_gen.py` tính giá trị này thông qua `np.linalg.cond(A, p=2)`.

Định lý sai số (Error Bound) **Định lý 3.1** [3]: Cho $Ax = b$ là hệ nghiệm đúng và $A\hat{x} = b + \delta b$ là hệ bị nhiễu (với δb là nhiễu không thể tránh khỏi do sai số làm tròn). Khi đó sai số tương đối của nghiệm bị chặn bởi:

$$\frac{\|\hat{x} - x\|}{\|x\|} \leq \kappa_2(A) \cdot \frac{\|\delta b\|}{\|b\|} \quad (13)$$

Ý nghĩa thực tiễn: $\kappa_2(A)$ đóng vai trò là hệ số khuếch đại sai số. Máy tính luôn có nhiễu làm tròn $\epsilon_{\text{machine}} \approx 10^{-16}$ (IEEE 754 [7]). Do đó sai số nghiệm tương đối có thể lên tới $\kappa_2(A) \cdot \epsilon_{\text{machine}}$. Khi $\kappa_2(A) \approx 10^{20}$, sai số nghiệm có thể đạt tới $10^{20} \times 10^{-16} = 10^4$ – tức là nghiệm hoàn toàn vô nghĩa.

Lưu ý quan trọng: Sai số tương đối trong bảng thực nghiệm là *relative residual* $\|A\hat{x} - b\|_2 / \|b\|_2$, khác với $\|\hat{x} - x\| / \|x\|$ trong Định lý 3.1. Residual nhỏ **không** đảm bảo nghiệm $\hat{x}$ gần đúng với x^* . Đây chính là “bẫy” đặc trưng của hệ ill-conditioned.

3.3.2 Thiết lập hai đối tượng khảo sát

Ma trận SPD – Well-conditioned Ma trận SPD được sinh bằng hàm `generate_spd_matrix` trong `data_gen.py`:

1. Sinh ma trận ngẫu nhiên $X \in \mathbb{R}^{n \times n}$, tính $A = XX^T$ (đảm bảo đối xứng, bán xác định dương).
2. Ép chéo trội nghiêm ngặt: với mỗi hàng i : $A[i][i] \leftarrow \sum_{j \neq i} |A[i][j]| + 1.0$.

Biến đổi ở bước 2 đảm bảo hai tính chất song song: (1) Gauss-Seidel chắc chắn hội tụ, và (2) số điều kiện $\kappa_2(A)$ giữ ở mức thấp (tỉ lệ tuyến tính với n).

Ma trận Hilbert H_n – Ill-conditioned Ma trận Hilbert là bài kiểm tra khắc nghiệt bậc nhất cho độ ổn định số học. Nó được định nghĩa bởi:

$$H_n[i][j] = \frac{1}{i+j+1}, \quad i, j = 0, 1, \dots, n-1 \quad (14)$$

(0-indexed). Ma trận Hilbert có $\kappa_2(H_n)$ tăng **theo hàm mũ** khi n tăng, và không thỏa mãn điều kiện chéo trội chặt hàng nên Gauss-Seidel không thể áp dụng được.

3.3.3 Kết quả đo lường sai số tương đối

Bảng 3 trình bày số điều kiện $\kappa_2(A)$ và sai số tương đối (relative residual) của ba phương pháp trên hai loại ma trận với $n \in \{50, 100, 200, 500\}$.

Bảng 3: So sánh số điều kiện $\kappa_2(A)$ và sai số tương đối $\|A\hat{x} - b\|_2 / \|b\|_2$

n	Loại	$\kappa_2(A)$	Gauss	SVD	Gauss-Seidel
50	Hilbert	1.10×10^{19}	5.58×10^{-16}	8.47×10^{-16}	ERR
	SPD	2.43	2.47×10^{-16}	7.83×10^{-16}	1.21×10^{-12}
100	Hilbert	3.31×10^{19}	5.81×10^{-15}	1.22×10^{-15}	ERR
	SPD	2.30	2.56×10^{-16}	8.35×10^{-16}	1.18×10^{-12}
200	Hilbert	1.60×10^{20}	2.67×10^{-15}	1.15×10^{-15}	ERR
	SPD	2.24	3.51×10^{-16}	1.74×10^{-15}	2.06×10^{-13}
500	Hilbert	4.76×10^{20}	2.49×10^{-14}	1.28×10^{-15}	ERR
	SPD	2.15	4.68×10^{-16}	1.37×10^{-15}	2.17×10^{-13}

ERR: Ma trận Hilbert không chéo trội $\rightarrow$ Gauss-Seidel từ chối giải.

3.3.4 Nhận xét và lý giải hiện tượng

Phân tích trên ma trận SPD (Well-conditioned) Trên ma trận SPD, $\kappa_2(A)$ rất nhỏ (từ 2.15 đến 2.43 – gần như bằng 1). Theo Định lý 3.1, sai số nghiệm bị chặn bởi $\kappa_2(A) \cdot \epsilon_{\text{machine}} \approx 2.5 \times 10^{-16}$. Kết quả thực nghiệm hoàn toàn nhất quán:

- **Gauss và SVD** cho residual luôn ở mức *machine epsilon* ($\sim 10^{-16}$ đến 10^{-15}), chứng tỏ cả hai đều cho kết quả chính xác đến giới hạn biểu diễn số dấu phẩy động.
- **Gauss-Seidel** hội tụ ổn định với sai số $\sim 10^{-12}$ đến 10^{-13} , ở mức chấp nhận được trong thực tiễn kỹ thuật. Sai số nhỉnh hơn so với hai phương pháp trực tiếp là do thuật toán lặp dừng khi $\|\Delta x\| < 10^{-10}$ chứ không giải chính xác.

Hiện tượng bùng nổ sai số (Error Explosion) trên ma trận Hilbert Trên ma trận Hilbert, $\kappa_2(H_n)$ đạt tới $\approx 4.76 \times 10^{20}$ (với $n = 500$). Áp dụng Định lý 3.1:

$$\frac{\|\hat{x} - x^*\|}{\|x^*\|} \lesssim \kappa_2(H_n) \cdot \varepsilon_{\text{machine}} \approx 4.76 \times 10^{20} \times 10^{-16} = 4.76 \times 10^4 \quad (15)$$

Điều này có nghĩa là nghiệm $\hat{x}$ tính được bởi Gauss có thể sai lệch tới **50,000 lần** so với nghiệm thật, mặc dù residual $\|A\hat{x} - b\|_2 / \|b\|_2$ nhìn có vẻ nhỏ (cỡ 10^{-15}). Đây chính là “bẫy” kinh điển của hệ ill-conditioned: **residual nhỏ không đồng nghĩa với nghiệm đúng**. Ma trận Hilbert “chấp nhận” mọi nghiệm xấp xỉ b với residual thấp, vì bản thân nó bị kéo dãn quá nhiều trong không gian nhiều chiều.

Sự ưu việt của SVD trên ma trận Hilbert Trên cùng ma trận Hilbert $n = 500$, SVD cho residual 1.28×10^{-15} trong khi Gauss cho 2.49×10^{-14} – thấp hơn Gauss khoảng **20 lần**. Lý do nằm ở bản chất của np.linalg.lstsq: nó sử dụng **nghịch đảo giả** (pseudo-inverse) dựa trên SVD, trong đó các giá trị suy biến $\sigma_i \approx 0$ (tương ứng với chiều không gian “phẳng” của Hilbert) được **cắt tỉa** (truncated) thay vì tính nghịch đảo $1/\sigma_i \rightarrow \infty$. Cơ chế regularization ngầm này loại bỏ hoàn toàn các thành phần gây khuếch đại nhiễu, làm cho SVD trở thành phương pháp ổn định nhất trên hệ ill-conditioned [2].

Hạn chế của Gauss-Seidel trên ma trận Hilbert Gauss-Seidel báo ERR (từ chối giải) với tất cả kích thước n trên ma trận Hilbert. Nguyên nhân: ma trận Hilbert không thỏa mãn điều kiện chéo trội chặt hàng. Về lý thuyết, bán kính phổ của ma trận lặp Gauss-Seidel $\rho(G_H) \geq 1$, khiến chuỗi lặp phân kỳ. Hàm check_diagonally_dominant phát hiện điều này ngay lập tức ở bước kiểm tra đầu vào, ném ValueError trước khi thực hiện bất kỳ vòng lặp nào – đây là thiết kế bảo vệ *fail-fast* có chủ đích.

3.4 Tổng kết và đánh giá lựa chọn phương pháp

Dựa trên các kết quả thực nghiệm và phân tích lý thuyết, nhóm rút ra bảng đánh giá tổng hợp sau:

- **Khử Gauss (LU):** Là phương pháp nhanh nhất và tiết kiệm bộ nhớ nhất trong ba phương pháp trực tiếp. Hằng số $\mathcal{O}(n^3)$ thấp nhất, phù hợp làm lựa chọn mặc định cho các hệ phương trình thông thường (well-conditioned). Điểm yếu duy nhất là hoàn toàn vô lực trước ma trận điều kiện kém: residual trông nhỏ nhưng nghiệm thực tế sai xa.
- **Phân rã SVD:** Đắt đỏ nhất về thời gian thực thi (chậm hơn Gauss $\approx 7\times$ ở $n = 1000$), nhưng sở hữu khả năng “miễn nhiễm” đáng kinh ngạc với sai số số học nhờ cơ chế nghịch đảo giả và cắt tỉa giá trị suy biến. Là **lựa chọn bắt buộc** cho các hệ ill-conditioned, bài toán bình phương tối thiểu, hoặc khi ma trận có thể suy biến (rank-deficient).

Bảng 4: Bảng so sánh tổng hợp ba phương pháp giải hệ phương trình tuyến tính

Tiêu chí	Khử Gauss (LU)	Phân rã SVD	Gauss-Seidel
Độ phức tạp	$O(n^3)$	$O(n^3)$ (hằng số lớn)	$O(kn^2)$ (k nhỏ)
Tốc độ ($n = 1000$)	0.036 s	0.253 s	1.09 s
Sai số / SPD	$\sim 10^{-16}$	$\sim 10^{-15}$	$\sim 10^{-12}$
Ổn định / Hilbert	Residual nhỏ, nghiệm sai	Tốt nhất	Không áp dụng
Điều kiện	Ma trận không suy biến	Mọi ma trận	Chéo trội chặt
Ứng dụng tối ưu	Hệ thông thường	Hệ ill-conditioned	Ma trận sparse/SDD

- **Lặp Gauss-Seidel:** Tốc độ tiệm cận $\mathcal{O}(n^2)$ – vượt trội so với hai phương pháp $\mathcal{O}(n^3)$ ở kích thước lớn. Tiết kiệm bộ nhớ (không cần lưu ma trận phân rã). Tuy nhiên, điều kiện áp dụng **rất khắt khe:** chỉ hoạt động với ma trận chéo trội nghiêm ngặt, và hoàn toàn thất bại trên Hilbert. Phù hợp nhất cho bài toán vật lý kỹ thuật tự nhiên có cấu trúc sparse với tính chéo trội (ví dụ: phương trình sai phân hữu hạn cho bài toán Poisson).

4 Kết luận

4.1 Tóm tắt kết quả đạt được

Kết thúc quá trình thực hiện đồ án, nhóm đã hoàn thành toàn diện 100% các mục tiêu đề ra ban đầu, bao gồm cả yêu cầu cốt lõi lẫn các phần mở rộng (điểm cộng). Các kết quả chính yếu bao gồm:

- **Cài đặt thành công Phép khử Gauss với độ ổn định cao:** Xây dựng thuật toán từ đầu (from scratch) bằng Python, áp dụng triệt để kỹ thuật Partial Pivoting. Thuật toán xử lý thành công mọi dạng ma trận (vuông, chữ nhật, suy biến) và đã được kiểm chứng tính ổn định số học tương đương với thư viện `numpy.linalg` thông qua việc triệt tiêu hiện tượng Swamping.
- **Phát triển hệ thống phân rã và chéo hóa ma trận (SVD):** Hoàn thiện thuật toán phân rã giá trị suy biến (Singular Value Decomposition) dựa trên nền tảng toán học của trị riêng và vector riêng. Thuật toán có khả năng phân rã và trích xuất đúng các ma trận U, Σ, V^T .
- **Xây dựng bộ công cụ phân tích hiệu năng (Benchmarking Suite):** Thiết lập thành công kịch bản đo lường tự động (`benchmark.py`) và hệ thống sinh dữ liệu (`data_gen.py`) để thu thập hàng ngàn điểm dữ liệu về thời gian thực thi và sai số tương đối (Residual Norm) trên đa dạng kích thước ma trận ($n \in \{50, 100, 200, 500, 1000\}$).
- **Phân tích sâu sắc tính bất ổn định số học:** Sử dụng thành công ma trận Hilbert H_n (hệ điều kiện kém) và ma trận SPD để bộc lộ hiện tượng khuếch đại sai số ở chuẩn dấu phẩy động IEEE 754. Qua đó, minh chứng rõ ràng mối liên hệ giữa Số điều kiện $\kappa_2(A)$ và mức độ "trôi dạt" của vector nghiệm.
- **Trực quan hóa Toán học sinh động bằng Manim:** Chuyển hóa các phương trình đại số tuyến tính khô khan thành Video hoạt hình (Animation) chất lượng cao. Video mô phỏng trực quan các phép biến đổi hình học của ma trận trong không gian, giúp giải thích trực diện bản chất của quá trình phân rã.
- **Quy trình phát triển phần mềm chuyên nghiệp:** Ứng dụng triệt để Git/GitHub trong quản lý mã nguồn nhóm, cấu trúc thư mục module hóa (`solvers`, `config`), kết hợp Jupyter Notebook và $\text{\LaTeX}$ để xuất bản báo cáo học thuật đạt chuẩn.
- Cài đặt thêm phương pháp lặp **Gauss-Seidel** với cơ chế tự động kiểm tra điều kiện hội tụ (ma trận chéo trội chặt hàng).
- Xây dựng kịch bản benchmark để đo lường thời gian thực thi trên các kích thước ma trận lớn ($n \in \{50, 100, 200, 500, 1000\}$). Vẽ đồ thị *log-log* so sánh và chứng minh thực nghiệm chi phí tính toán của các phương pháp trực tiếp hoàn toàn bám sát đường lý thuyết $O(n^3)$.
- Nghiên cứu thành công sự bất ổn định số học: Sử dụng **Số điều kiện $\kappa_2(A)$** kết hợp đối chiếu

giữa ma trận Hilbert H_n (ill-conditioned) và ma trận SPD ngẫu nhiên để bộc lộ hiện tượng khuếch đại sai số, giải thích sâu sắc giới hạn của hệ thống dấu phẩy động IEEE 754.

4.2 Bài học rút ra

Quá trình thực hiện đồ án không chỉ là việc hiện thực hóa các công thức toán học lên ngôn ngữ lập trình, mà còn là một hành trình giúp nhóm thay đổi tư duy về khoa học tính toán:

- **Sự khác biệt giữa Toán học lý thuyết và Tính toán số học:** Nhóm nhận ra rằng một thuật toán đúng trên giấy chưa chắc đã chạy đúng trên máy tính. Việc hiểu về cấu trúc lưu trữ số thực *double precision* là chìa khóa để giải thích tại sao $0.1 + 0.2$ không bao giờ bằng 0.3 tuyệt đối trong RAM.
- **Tư duy phản biện qua dữ liệu:** Thay vì chấp nhận sai số, nhóm đã học được cách đặt câu hỏi “Tại sao?”. Việc quan sát đường sai số bám sát ngưỡng 10^{-16} đã giúp nhóm tin tưởng vào thuật toán của mình hơn là việc chỉ nhìn vào kết quả nghiệm cuối cùng.
- **Kỹ năng tối ưu hóa quy trình làm việc:** Nhóm đã làm quen với việc tự động hóa (Automation) thông qua *latexmk* và các script *benchmark*. Bài học về việc “lười biếng một cách thông minh” – viết script để máy tính tự đo đạc hàng ngàn phép tính – đã giúp nhóm tiết kiệm rất nhiều thời gian.
- **Giá trị của việc chuẩn hóa (Standardization):** Việc thống nhất sử dụng `config.py` và quy chuẩn `list[list[float]]` ngay từ đầu là bài học xương máu về quản lý dự án, giúp việc ghép code giữa các thành viên diễn ra trơn tru mà không bị xung đột kiểu dữ liệu.

4.3 Khó khăn chuyên môn và các nghịch lý Toán học tính toán

Trong suốt quá trình thực hiện đồ án, rào cản lớn nhất của nhóm không nằm ở cú pháp lập trình, mà nằm ở sự sai khác giữa Toán học lý thuyết (trên giấy) và Giải tích số (trên máy tính). Dưới đây là bốn khó khăn cốt lõi và bài học nhận thức nhóm đã rút ra.

1. Giới hạn của chuẩn IEEE 754

Hiện tượng: Trong giai đoạn kiểm thử ban đầu, nhóm kỳ vọng sai số phần dư sẽ bằng 0.0 tuyệt đối. Tuy nhiên, khi giải các hệ phương trình chứa phân số (như $\frac{1}{3}$) hay số thập phân (như 0.1), sai số phần dư luôn trôi nổi ở mức 10^{-16} .

Bản chất Toán học: Đây không phải là lỗi thuật toán, mà là giới hạn biểu diễn tất yếu của chuẩn dấu phẩy động IEEE 754 (Double Precision) [7]. Các phân số vô hạn tuần hoàn trong hệ cơ số 2 bị cắt cụt ở bit thứ 53, sinh ra nhiễu làm tròn (round-off error). Ranh giới này được gọi là *Machine Epsilon*: $\epsilon_{\text{machine}} \approx 2.22 \times 10^{-16}$.

Giải quyết: Nhóm phải từ bỏ tư duy ”so sánh bằng 0” (exact equality) của Toán học. Thay vào đó, hằng số $\text{EPSILON} = 1\text{e-}12$ được thiết lập tập trung trong `config.py` làm ngưỡng dung sai (tolerance), cho phép các thuật toán phân rã SVD và Jacobi hội tụ một cách an toàn mà không bị ảnh hưởng bởi nhiễu máy tính.

2. Sai số phần dư trên ma trận Hilbert

Hiện tượng: Khi đưa ma trận Hilbert H_n (với $n \geq 50$) vào thuật toán Gauss, máy tính trả về vector nghiệm $\hat{x}$ sai lệch hoàn toàn so với nghiệm lý thuyết. Điều kỳ lạ là khi kiểm tra lại bằng sai số phần dư $\|A\hat{x} - b\|_2$, kết quả vẫn cực kỳ nhỏ (cỡ 10^{-15}) – máy tính báo “đúng” nhưng nghiệm thực tế lại “sai hoàn toàn”.

Bản chất Toán học: Nhóm đã vấp phải nghịch lý kinh điển của hệ điều kiện kém (ill-conditioned system). Ma trận Hilbert có số điều kiện $\kappa_2(A)$ khổng lồ, lên tới 10^{20} . Theo Định lý sai số [3]:

$$\frac{\|\Delta x\|}{\|x\|} \leq \kappa_2(A) \cdot \frac{\|\Delta b\|}{\|b\|}$$

Chỉ cần một hạt nhiễu $\epsilon_{\text{machine}}$ rất yếu trong vế phải b , sai số thực tế (forward error) của nghiệm sẽ bị khuếch đại lên $\kappa_2(A)$ lần. Sai số phần dư nhỏ chỉ là một ảo ảnh toán học do không gian bị kéo dãn quá mức – đây là “bẫy” đặc trưng của hệ ill-conditioned mà bất kỳ nhà giải tích số nào cũng phải cảnh giác.

Giải quyết: Sự kiện này buộc nhóm phải nhìn nhận lại giá trị thực sự của thuật toán SVD. Nhờ khả năng dùng nghịch đảo giả (pseudo-inverse) và cắt tỉa các giá trị suy biến tiệm cận 0, SVD đã chứng minh được đây là công cụ duy nhất “miễn nhiễm” một phần với sự bùng nổ sai số trên hệ điều kiện kém.

3. Bức tường của hằng số $\mathcal{O}(n^3)$ và nút thắt hiệu năng Python

Hiện tượng: Trong lý thuyết, cả Khử Gauss và SVD đều có độ phức tạp $\mathcal{O}(n^3)$. Tuy nhiên, khi nhóm chạy benchmark bằng Python thuần ở $n = 500, 1000$, chương trình bị treo (Timeout > 60 s). Trong khi đó, `np.linalg.solve` của NumPy giải quyết $n = 1000$ chỉ trong 0.036 giây.

Bản chất Toán học và Kiến trúc: Độ phức tạp $\mathcal{O}(n^3)$ chỉ mô tả tốc độ tăng trưởng – nó che giấu một hằng số thực thi C rất lớn. Thuật toán SVD (Jacobi/QR) đòi hỏi nhiều vòng quét, phép tính lượng giác và căn bậc hai hơn hẳn Gauss, khiến $C_{\text{SVD}} \gg C_{\text{Gauss}}$. Bên cạnh đó, rào cản của trình thông dịch Python (interpreted) khiến phần cứng không thể song song hóa các phép nhân ma trận để phát huy tối đa khả năng của CPU.

Giải quyết: Dưới sự cho phép của giảng viên, nhóm đã linh hoạt chuyển sang sử dụng backend BLAS/LAPACK thông qua NumPy cho các kích thước n lớn. Quyết định này đảm bảo thu thập đủ số liệu để vẽ đồ thị Log-Log chuẩn xác và minh chứng được sự song song giữa đường thực nghiệm với đường tiệm cận $\mathcal{O}(n^3)$ lý thuyết.

4. Ranh giới hội tụ khắt khe của phương pháp lặp Gauss-Seidel

Hiện tượng: Ban đầu, khi thử nghiệm Gauss-Seidel trên các ma trận SPD ngẫu nhiên, thuật toán liên tục báo ERR (vượt quá số vòng lặp tối đa mà không hội tụ), mặc dù lý thuyết khẳng định Gauss-Seidel chắc chắn hội tụ trên ma trận SPD.

Bản chất Toán học: Định lý chỉ khẳng định ma trận SPD sẽ hội tụ, nhưng không hứa hẹn tốc độ hội tụ. Nếu bán kính phổ (spectral radius) của ma trận lặp $\rho(G)$ nằm quá sát 1, phương pháp lặp tiến triển cực kỳ chậm và cạn kiệt giới hạn vòng lặp cho phép trước khi đạt được ngưỡng hội tụ.

Giải quyết: Nhóm đã can thiệp vào khâu sinh dữ liệu (`data_gen.py`), bắt buộc mọi ma trận SPD phải được cộng thêm biên độ vào đường chéo chính để thỏa mãn điều kiện **chéo trội nghiêm ngặt** (Strictly Diagonally Dominant):

$$|a_{ii}| > \sum_{j \neq i} |a_{ij}|, \quad \forall i = 1, \dots, n$$

Chỉ khi ép chặt điều kiện này, bán kính phổ $\rho(G)$ mới thực sự nhỏ hơn 1 đủ nhiều, giúp Gauss-Seidel hội tụ ổn định và phát huy sức mạnh tốc độ $\mathcal{O}(n^2)$ đặc trưng của phương pháp lặp.

Tài liệu

- [1] Gilbert Strang, *Introduction to Linear Algebra*, 5th Edition, Wellesley-Cambridge Press, 2016.
- [2] Gene H. Golub và Charles F. Van Loan, *Matrix Computations*, 4th Edition, Johns Hopkins University Press, 2013.
- [3] Lloyd N. Trefethen và David Bau III, *Numerical Linear Algebra*, SIAM, 1997.
- [4] Virginia C. Klema và Alan J. Laub, *The singular value decomposition: Its computation and some applications*, IEEE Transactions on Automatic Control, Vol. 25, No. 2, pp. 164–176, 1980.
- [5] IEEE Computer Society, *IEEE Standard for Floating-Point Arithmetic (IEEE 754-2019)*, <https://ieeexplore.ieee.org/document/8766229>, truy cập ngày 09/04/2026.
- [6] NumPy Developers, *NumPy v1.26 Manual – Linear Algebra (`numpy.linalg`)*, <https://numpy.org/doc/stable/reference/routines.linalg.html>, truy cập ngày 09/04/2026.
- [7] Manim Community Developers, *Manim Community Documentation v0.18.0*, <https://docs.manim.community/>, truy cập ngày 09/04/2026.

A Phụ lục

A.1 Bảng số liệu chi tiết

Dưới đây là bảng số liệu chi tiết thu thập được trong quá trình thực nghiệm:

STT	Tham số	Giá trị đo được
1	Tham số A	10.5

Bảng 5: Bảng số liệu chi tiết

A.2 Code bổ sung

Đoạn mã nguồn quan trọng được sử dụng trong hệ thống:

```
1 import numpy as np
2
3 def process_data(data_path):
4     print("Code minh hoa chen tai phu luc...")
5     return True
```

Đoạn mã 9: Mã nguồn xử lý dữ liệu (Python)